

# DBSCAN Density Clustering: Numerical Walkthrough

**DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** is one of the most important clustering algorithms because, unlike K-Means, you do not need to specify the number of clusters ( $K$ ) in advance, and it is capable of discovering clusters of arbitrary shapes while cleanly filtering out outliers.

**The Core Philosophy:** Points that are in dense regions belong to the same cluster. Points that are isolated are treated as noise (outliers).

## Why DBSCAN? (K-Means vs. DBSCAN)

Consider a dataset with two clusters and one isolated point (outlier):

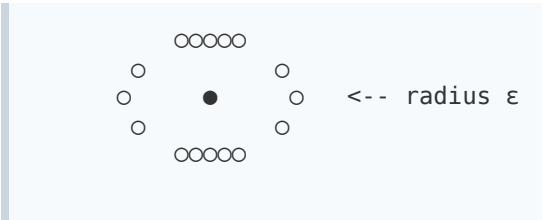


- **K-Means Problem:** If you choose  $K = 2$ , K-Means will force every point into one of the two clusters, dragging the cluster centers and assigning the isolated outlier ( $\times$ ) to one of them.
- **DBSCAN Solution:** DBSCAN recognizes that the isolated point doesn't belong to any dense neighborhood and labels it as **Noise**.

## DBSCAN Parameters

DBSCAN relies on two key parameters to define density:

1.  **$\epsilon$  (Epsilon):** The radius around a point defining its search neighborhood. Imagine drawing a circle of radius  $\epsilon$  around every point.
2. **MinPts:** The minimum number of points required inside the  $\epsilon$ -radius (including the point itself) to call that region "dense".



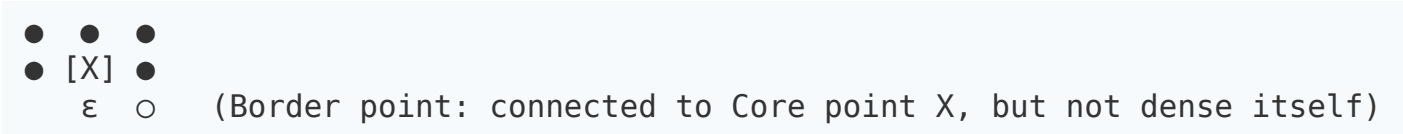
Three Types of Points

DBSCAN classifies every data point into one of three categories:

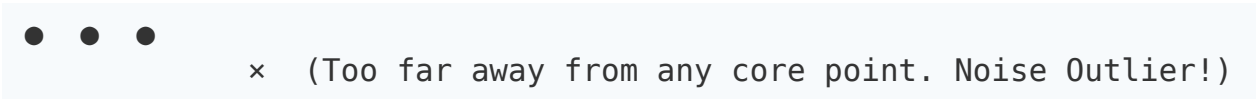
- 1. **Core Point:** A point with at least `MinPts` neighbors inside its  $\epsilon$ -neighborhood.



- 2. **Border Point:** A point that has fewer than `MinPts` neighbors, but falls within the  $\epsilon$ -neighborhood of a core point.



- 3. **Noise Point:** A point with too few neighbors that is not reachable from any core point.



Mathematical Definition

Let the  $\epsilon$ -neighborhood of a point  $p$  be defined as:  $N_{\epsilon}(p) = \{q \mid \text{distance}(p,q) \leq \epsilon\}$  A point  $p$  is a **Core Point** if:  $|N_{\epsilon}(p)| \geq \text{MinPts}$  Otherwise, if  $p$  is in  $N_{\epsilon}(c)$  for some core point  $c$ , it is a **Border Point**. Otherwise, it is a **Noise Point**.

The Sample Dataset

Imagine we have 7 points plotted on a 2D coordinate grid:

| Point | X Coordinate | Y Coordinate |
|-------|--------------|--------------|
| A     | 1            | 1            |
| B     | 1            | 2            |
| C     | 2            | 2            |

| Point | X Coordinate | Y Coordinate |
|-------|--------------|--------------|
| D     | 2            | 1            |
| E     | 8            | 8            |
| F     | 8            | 9            |
| G     | 20           | 20           |

We configure DBSCAN with  $\epsilon = 2$  and **MinPts** = 3.

## Step-by-Step Numerical Scan

### Step 1: Process Point A (1, 1)

1. Find all neighbors within  $\epsilon = 2$  using Euclidean distance. Neighbors are {A, B, C, D}.
2. Count is 4. Since  $4 \geq \text{MinPts} (3)$ , **A is a Core Point**.
3. Start **Cluster 1**:  $\{A\}$ . Add A's neighbors  $\{B, C, D\}$  to the expansion queue.

### Step 2: Expand Cluster 1

We pull neighbors from the queue one by one and check if they can expand the cluster:

- **Process B (1, 2):** Neighbors within radius 2 are {A, B, C, D} (count  $4 \geq 3$ ). B is a Core Point! B joins Cluster 1. Since its neighbors are already known, no new points are queued.
- **Process C (2, 2):** Neighbors are {A, B, C, D} (count  $4 \geq 3$ ). C is a Core Point! C joins Cluster 1.
- **Process D (2, 1):** Neighbors are {A, B, C, D} (count  $4 \geq 3$ ). D is a Core Point! D joins Cluster 1.

Cluster 1 expansion is complete. **Cluster 1** = {A, B, C, D}.

### Step 3: Process Point E (8, 8)

1. Select E (next unvisited point). Neighbors are {E, F} (distance is 1.0).
2. Count is 2. Since  $2 < \text{MinPts} (3)$ , E is not a core point. It is temporarily marked as Noise.

### Step 4: Process Point F (8, 9)

1. Select F. Neighbors are {E, F}. Count is 2. Since  $2 < \text{MinPts} (3)$ , F is not a core point.
2. Since neither E nor F is reachable from any core point, they remain unconnected.

### Step 5: Process Point G (20, 20)

1. Neighbors of G is only {G}. Count is 1. Not a core point.
2. Marked as Noise.

Detailed Summary Chart

| Point | Coordinates | Neighbors ( $\epsilon=2$ ) | Point Class     | Final Assignment |
|-------|-------------|----------------------------|-----------------|------------------|
| A     | (1, 1)      | 4                          | Core Point      | Cluster 1        |
| B     | (1, 2)      | 4                          | Core Point      | Cluster 1        |
| C     | (2, 2)      | 4                          | Core Point      | Cluster 1        |
| D     | (2, 1)      | 4                          | Core Point      | Cluster 1        |
| E     | (8, 8)      | 2                          | Noise / Outlier | Noise (Isolated) |
| F     | (8, 9)      | 2                          | Noise / Outlier | Noise (Isolated) |
| G     | (20, 20)    | 1                          | Noise / Outlier | Noise (Outlier)  |

## Deep Dive: Understanding "Expand Cluster" & Queue Expansion

The heart of DBSCAN is the **Expand Cluster** phase. It operates like a chain reaction (similar to **Breadth-First Search (BFS)** in graph theory). Once a starting core point is found, the cluster "spreads" to all reachable neighbors. If a neighbor is also a core point, it spreads the cluster to *its* neighbors, growing the cluster through connected dense regions.

**The Spilling Water Analogy:** Think of it like spilling water. Initially, water hits the starting core point. The water then spreads to all its neighbors. If those neighbors are also core points, they spread the water even further, creating a continuous flow until the water hits a boundary (border points) or runs out of connected dense points.

### The Queue-Based Chain Reaction Example

Let's trace this expansion meticulously using a queue. Suppose we have the following 8 points:

```

A  B  C
D  E  F
    G
          H

```

Coordinates: **A(1,3), B(2,3), C(3,3), D(1,2), E(2,2), F(3,2), G(3,1), H(10,10).**

We configure  $\epsilon = 1.5$  and **MinPts** = 3.

#### 1. Step 1: Pick Point A

A's neighbors within radius 1.5 are **{A, B, D, E}** (count  $4 \geq 3$ ). A is a Core Point!

Start **Cluster 1** = **{A}**. Initialize our search **Queue** = **[B, D, E]**.

#### 2. Step 2: Pop B from Queue

B's neighbors are **{A, B, C, D, E, F}** (count  $6 \geq 3$ ). B is a Core Point!

Add B to Cluster 1: **Cluster 1** = **{A, B}**.

Since B is a Core Point, add its unvisited neighbors to the queue. **Queue** = **[D, E, C, F]**. (Note: C and F are newly discovered!).

#### 3. Step 3: Pop D from Queue

D's neighbors are **{A, B, D, E}** (count  $4 \geq 3$ ). D is a Core Point!

Add D to Cluster 1: **Cluster 1** = **{A, B, D}**.

No new unvisited neighbors to add. **Queue** = **[E, C, F]**.

#### 4. Step 4: Pop E from Queue

E's neighbors are **{A, B, C, D, E, F}** (count  $6 \geq 3$ ). E is a Core Point!

Add E to Cluster 1: **Cluster 1** = **{A, B, D, E}**.

No new unvisited neighbors. **Queue** = **[C, F]**.

**5. Step 5: Pop C from Queue**

C's neighbors are {**B, C, E, F, G**} (count  $5 \geq 3$ ). C is a Core Point!

Add C to Cluster 1: **Cluster 1** = {**A, B, D, E, C**}.

Add C's unvisited neighbor G to the queue. **Queue** = [**F, G**].

**6. Step 6: Pop F from Queue**

F's neighbors are {**B, C, E, F, G**} (count  $5 \geq 3$ ). F is a Core Point!

Add F to Cluster 1: **Cluster 1** = {**A, B, D, E, C, F**}.

No new unvisited neighbors. **Queue** = [**G**].

**7. Step 7: Pop G from Queue**

G's neighbors are {**C, F, G**} (count  $3 \geq 3$ ). G is a Core Point!

Add G to Cluster 1: **Cluster 1** = {**A, B, D, E, C, F, G**}.

No new unvisited neighbors. **Queue** = [] (Queue is empty!).

The queue is empty, so expansion stops. Point H (10, 10) is too far away to ever be reached and is labeled as **Noise**.

**Crucial Insight: Why do we still check D, E, etc., if they added no new points?**

Because we don't know in advance whether they have additional neighbors! If the data had a branch extending down from D (like another point at (1, 0.5)), skipping D would mean missing that entire branch, leaving the cluster incomplete. Every point added to the cluster must be processed exactly once to guarantee we find every density-connected point.

**DBSCAN Expansion Pseudocode**

```
algorithm ExpandCluster(dataset, point, neighbors, cluster_id, eps, min_pts) is
    assign point to cluster_id
    seeds := queue(neighbors)

    while seeds is not empty do
        current_point := seeds.pop()

        if current_point is unvisited then
            mark current_point as visited
            current_neighbors := FindNeighbors(dataset, current_point, eps)

            if length(current_neighbors) ≥ min_pts then
                seeds.push_all(current_neighbors)

        if current_point is not assigned to any cluster then
            assign current_point to cluster_id
```

**Difference Between K-Means and DBSCAN**

| Feature            | K-Means Clustering                            | DBSCAN Clustering                                      |
|--------------------|-----------------------------------------------|--------------------------------------------------------|
| Number of Clusters | Must specify $K$ in advance                   | Discovered automatically based on density              |
| Core Mechanism     | Uses centroids (cluster means)                | Uses density (neighbor count within $\epsilon$ radius) |
| Outlier Handling   | Forces every point into a cluster (sensitive) | Naturally detects and isolates noise points            |
| Cluster Shapes     | Assumes spherical/globular clusters           | Finds arbitrary, complex, and winding shapes           |
| Key Parameters     | $K$ (Number of Clusters)                      | $\epsilon$ (Search Radius) & MinPts (Density Limit)    |

Real-World Applications

- **Credit Card Fraud Detection:** Grouping normal transactions together while isolating fraudulent transactions as noise.
- **GPS Trajectory Analysis:** Clustering coordinates to identify frequently visited hotspots or routes.
- **Earthquake Epicenter Clustering:** Detecting seismic fault lines by grouping dense clusters of earthquake coordinates.
- **Customer Segmentation with Outliers:** Clustering core customer segments without letting spam accounts or extreme spenders distort the groups.

Interview Spotlight: "What is DBSCAN?"

"DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised density-based clustering algorithm that groups points located in close proximity (dense regions) while labeling isolated points in sparse regions as noise. It requires two main parameters:  $\epsilon$  (neighborhood radius) and MinPts (density threshold). Unlike K-Means, DBSCAN does not require specifying the number of clusters in advance, can discover clusters of arbitrary shapes (like rings or spirals), and is highly robust to outliers, which it automatically identifies."

Unsupervised Learning Masterclass • © 2026 Visual Explorer Project